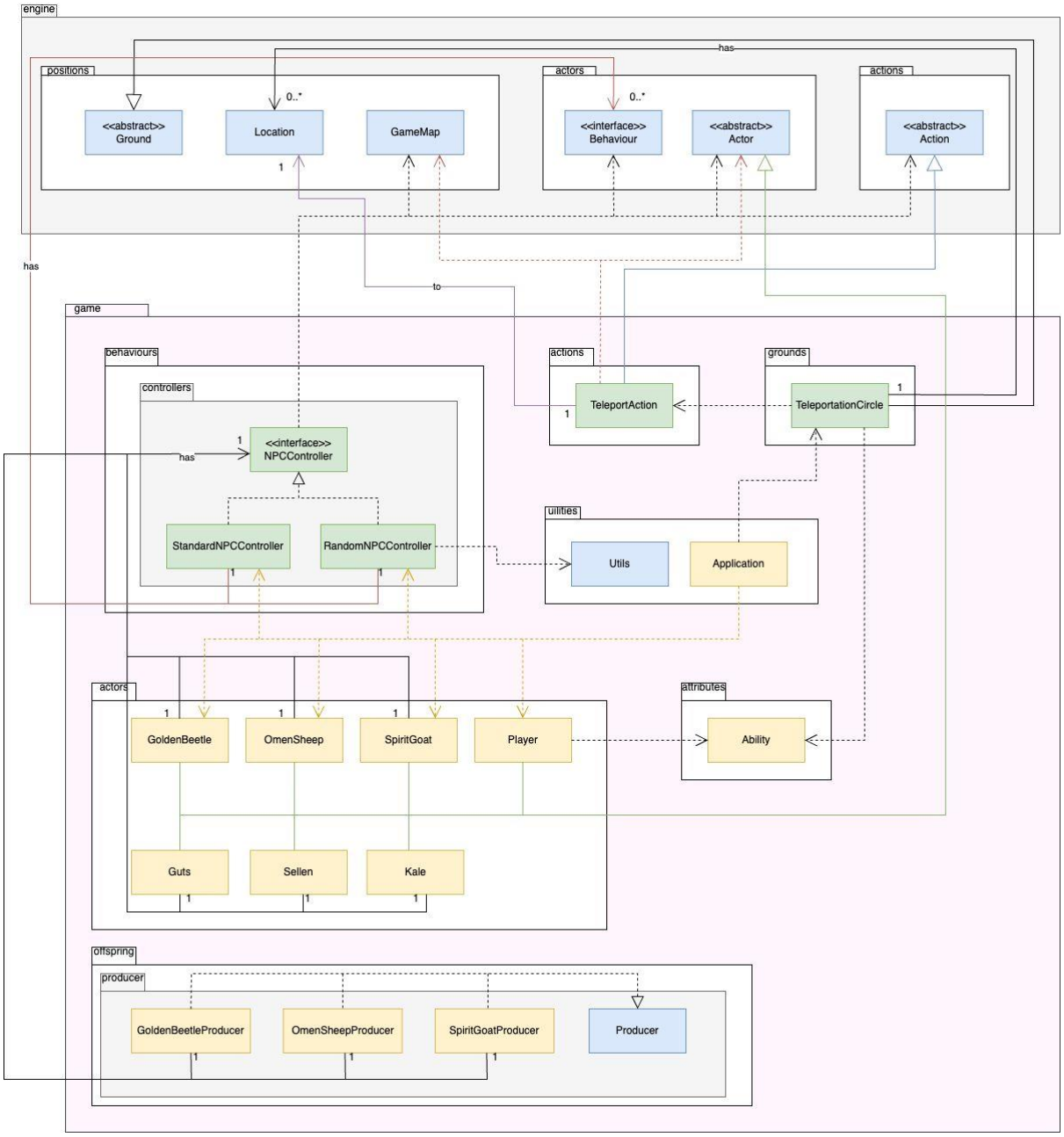
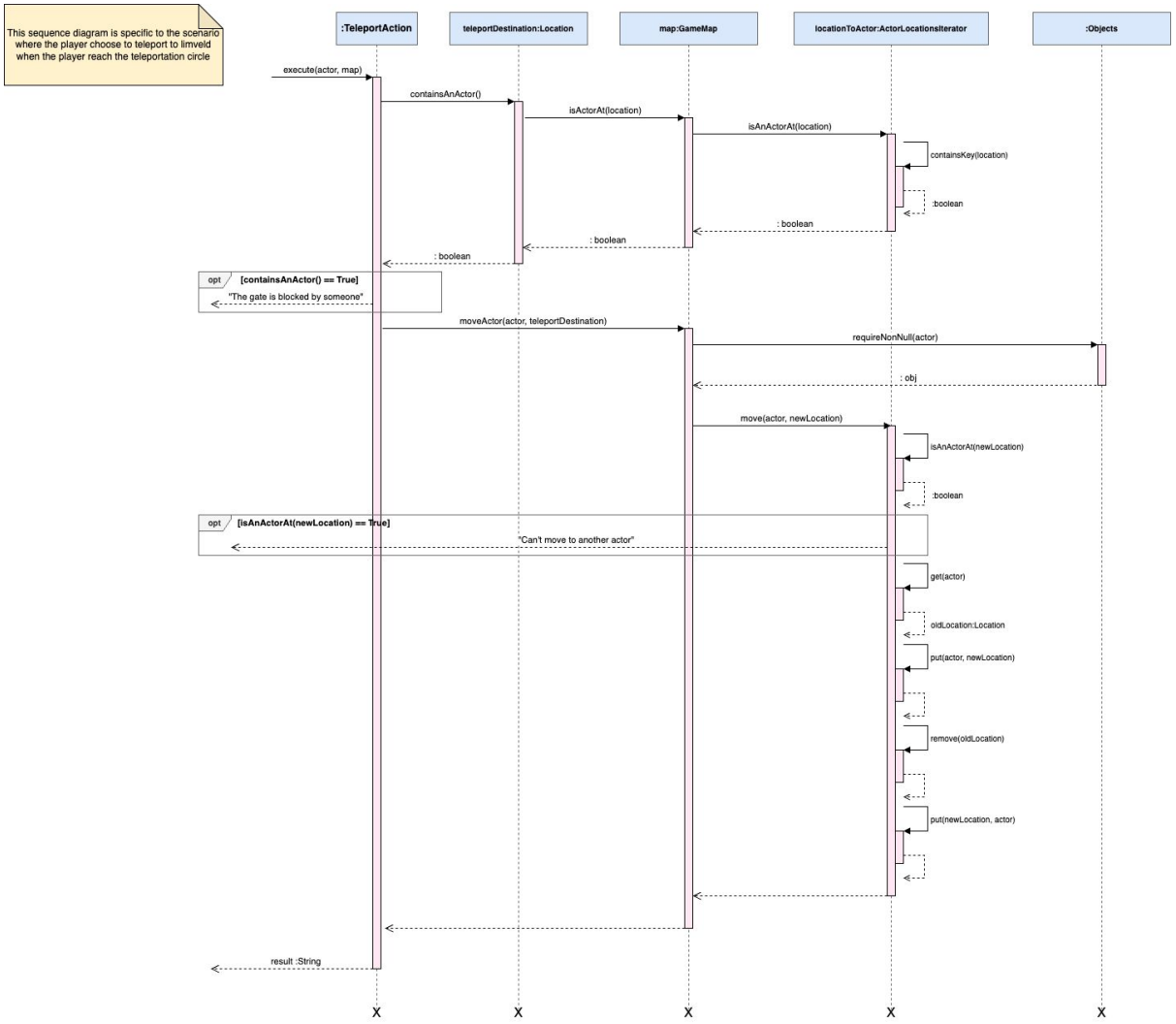
REQ1

UML Diagram:

Colour	Indicator
Yellow	Modified class
Blue	Existing class
Green	New class



Sequence Diagram:



A. Removal of NPC abstract class and added NPCController interface as replacement

Introduced NPCController interface

Interface method:

performTurn: define the logic of how the NPC should perform in each turn.

addBehaviour: define the logic of how the behaviour should be added

createNew: to produce the same controller object for the offspring

Pros and Cons of NPCController interface

Pros	**Cons**
Allows each Controller to encapsulate the own logic of how NPC behaves.	High cost to refactor interface if new interface method is added when there are a lot of controller introduced

Promote low coupling. Reduce dependencies when classes needed a NPCController.	

Design 1:

Keep the NPC abstract class together with the NPCController interface.

Pros	**Cons**
Common methods or field can be centralized in the abstract class.	Tight coupling.
	Violates OCP if NPC base class required refactor for new NPC to be introduced, if the NPC did not fit the abstraction.

Design 2:

Remove the NPC abstract class but added a new NPCController interface. Make all the previous class which extends from NPC abstract class to extend from Actor abstract class and implements NPCController interface.

Pros	**Cons**
New NPC can be added without considering limitations from NPC abstract class.	Loss of centralized control.
Allow encapsulation. Each NPC only handles what they need. Each different functionality and responsibilities can be handled by interface but not abstraction.	

B. Added RandomNPCController and StandardNPCController which implements NPCController interface

Pros	**Cons**
Promotes low coupling. Reduce dependencies. Controller can be used anywhere when a NPCController is expected.	Each new behaviour controller requires a new NPCController class, which increase code management.

Promotes encapsulation. Encapsulate each different behaviour controlling logic in one class.	

C. Added TeleportationCircle concrete class that extends from Ground

Pros	**Cons**
Encapsulates all teleport specific ground logic in one class	
Easier to extend or modify teleport related logic.	

D. Added TELEPORT_ABILITY enum

Pros	**Cons**
Promotes clarity and consistency in checking teleport eligibility.	Increase complexity in Actor capability management
Easy extensible as any new Actor that can teleport can be added just by adding this capability enum.	

E. Use character 'A' as constant to represent the teleport circle symbol

Pros	**Cons**
Clear separation from other symbol being used across the application which can be easily identified.	Might be unclear to users without context.
Easy to debug using map output.	

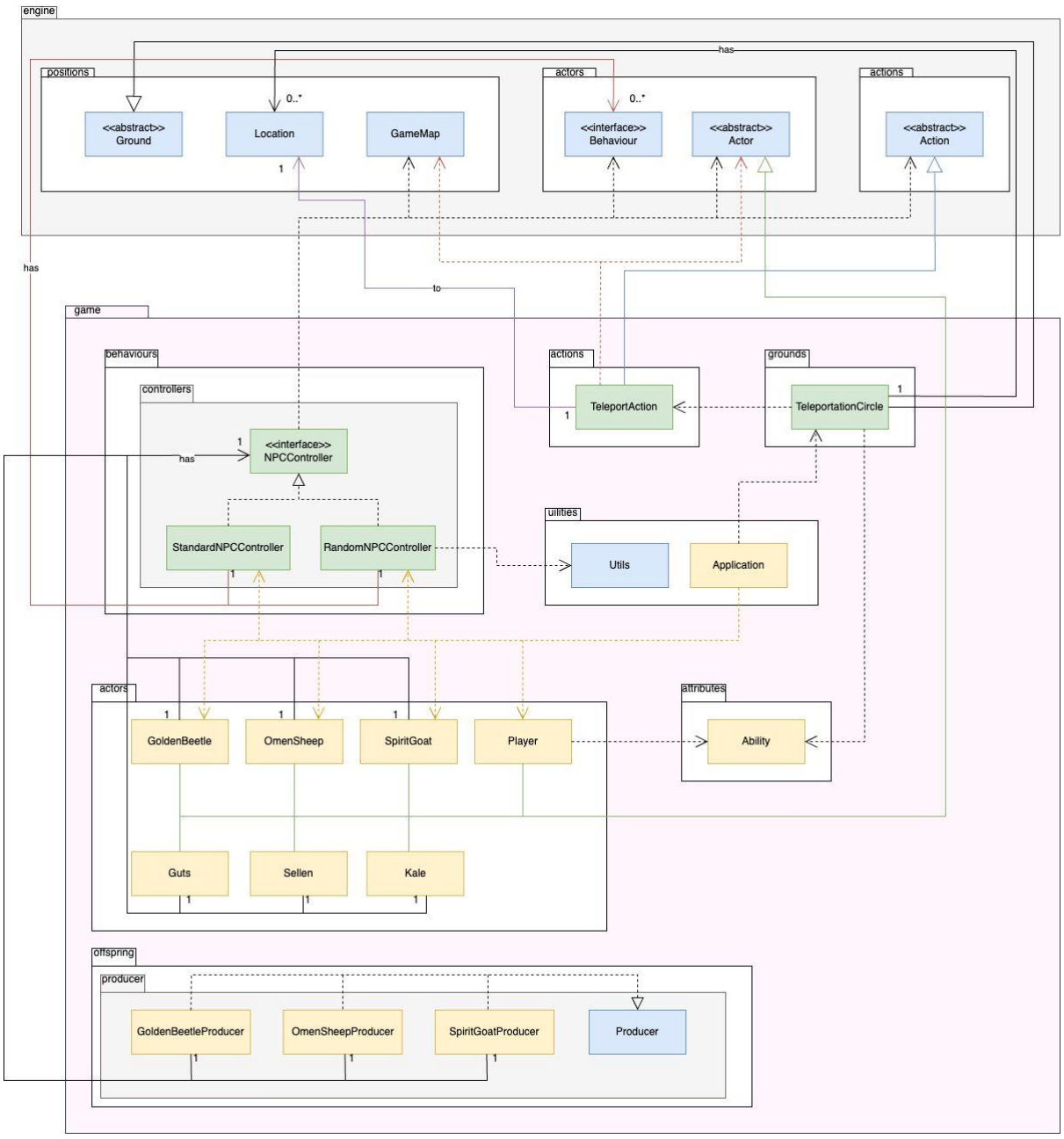
F. Added TeleportAction class which extends from Action abstract class

Pros	**Cons**

Promotes encapsulation. Encapsulate the related executing teleportation logic in one class.	High cost for Action class refactoring if each Action related class extends from Action class.
Promotes extension. Allow any Actor with capability of TELEPORT_ABILITY to perform this action.	

G. Added NPCController attribute to all NPC (previous extends from NPC abstract class) and all Producer class who produce an Actor

Pros	**Cons**
Flexibility in choosing NPCController for the NPC	
Avoid encapsulate logic related to choosing behaviour in class themselves. Separate responsibilities.	



Above shows the uml diagram which follows design A (design 2), B, C, D, E, F and G.

Create a NPCController interface with performTurn, addBehaviour, and createNew method. This allows each Controller to define their own logic of how the NPC should be controlled such as how the NPC should perform each turn (SRP). This also promotes OCP since it allows different controller to introduced in the future to control the NPC (OCP). Every controller can be substituted into anywhere when a NPCController is expected (LSP and DIP).

Modified every constructor of each NPC to accept a new NPCController parameter and added a NPCController attribute to each NPC. This allows each NPC to accept any Controller that implements the NPCController to define how to perform each turn (OCP).

Create RandomNPCController and StandardNPCController which implements the NPCController interface. This allows each Controller to define their own controller logic in their own class (SRP) and able to substitute in anywhere when a NPCController is expected (LSP) where it allows any current or new NPC with a NPCController attribute to use (OCP).

Remove NPC abstract class and let all previous class that extends from NPC abstract class to extend from Actor abstract class. This promotes OCP as new NPC can be added by extending from Actor without considering the limitations from the NPC abstract class, where specific functionality can be handled by interface. It also allows the responsibilities are more clearly divided where specific functionality or behaviour of NPC can be controlled by interface rather than abstract class (SRP).

Added NPCController attribute to all NPC class and Producer class. This allows each NPC or Producer to have any NPCController based on the requirements (OCP). This also allow the logic related of controlling an NPC action to encapsulate in the respective NPCController class but not the NPC class itselves (SRP). This also promotes DIP since it will reduce dependencies but only depends on the NPCController interface itself while able to access to different types of Controller.

Added TeleportationCircle class which extends from Ground. This allows the teleport circle be able to add into anywhere of the map as long as it supports a Ground (LSP). It also encapsulates the logic related to the teloportation ground tiles such as the display character and the location to be teleport (SRP). With ArrayList of destination as attributes, it supports further location to be teleport in the future (OCP).

Added TeleportAction class which extends from Action abstract class. This allows any Actor with TELEPORT_ABILITY can perform this action by simply adding TELEPORT_ABILITY enum (OCP). This class also encapsulates all the logic of executing the teleport related logic (SRP).